On-device edge learning for IoT data streams: a survey

Afonso Lourenço[a,*], João Rodrigo[a], João Gama[b] and Goreti Marreiros[a]

[a]GECAD, ISEP, Polytechnic of Porto, Rua Dr. António Bernardino de Almeida, Porto, 4249-015, Portugal
[b]INESC TEC, FEP, University of Porto, Rua Dr. Roberto Frias, Porto, 4200-465, Portugal

ARTICLE INFO

Keywords:
Internet of Things
edge computing
ubiquitous computing
online learning
continual learning
data stream mining

ABSTRACT

This literature review explores continual learning methods for on-device training in the context of neural networks (NNs) and decision trees (DTs) for classification tasks on smart environments. We highlight key constraints, such as data architecture (batch vs. stream) and network capacity (cloud vs. edge), which impact TinyML algorithm design, due to the uncontrolled natural arrival of data streams. The survey details the challenges of deploying deep learners on resource-constrained edge devices, including catastrophic forgetting, data inefficiency, and the difficulty of handling IoT tabular data in open-world settings. While decision trees are more memory-efficient for on-device training, they are limited in expressiveness, requiring dynamic adaptations, like pruning and meta-learning, to handle complex patterns and concept drifts. We emphasize the importance of multi-criteria performance evaluation tailored to edge applications, which assess both output-based and internal representation metrics. The key challenge lies in integrating these building blocks into autonomous online systems, taking into account stability-plasticity trade-offs, forward-backward transfer, and model convergence.

arXiv:2502.17788v1 [cs.LG] 25 Feb 2025

1. Introduction

In today's interconnected world, nearly every electronic device is transmitting data over the internet, whether intentionally or not. The Internet of Things (IoT) continues to evolve, enabling the optimization of processes across a wide range of domains [144]. While initially, only servers had the necessary computing power for advanced analytics, as technology evolved, smaller devices had competing power for some applications, eliminating network delays in areas where critical decisions must be made in an instant. This shift in data generation and utilization gives rise to two key paradigms: ubiquitous computing, which refers to the pervasive presence of processing power throughout our environments, making them more interconnected and intelligent; and edge computing, which emphasizes the location of data processing by moving computation closer to the data source, reducing reliance on centralized cloud infrastructures. In particular, due to the widespread adoption of relational databases in these domains, tabular data is the dominant modality in these IoT applications. Organized into rows and columns, consisting of distinct features that are typically continuous, categorical, or ordinal, data arrives continuously as an infinite data stream.

Lifelong learning. This challenges the traditional batch learning paradigm, which assumes the availability of all training data upfront and its independent and identically distributed nature. Since a model only has access to the current data in an individual phase of the learning cycle, it is prone to overfit on the currently available data, with catastrophic forgetting of previous data [126]. Conversely, holding on to outdated knowledge can hinder the model ability to adapt fast and effectively learn from new data. Naturally, this stability-plasticity dilemma, is related to the challenge of learning invariant representations where the model exhibits both forward and backward transfer, in which learning a new concept not only takes advantage but also benefits from the knowledge extracted from old concepts.

Resource efficiency. Simultaneously, another challenge comes from the practical memory constraint of handling very long non-stationary sequences, without storing data. Under such a constraint, one must develop effective surrogate learning objectives that can account for past errors with high resource efficiency, quantified through computational overhead or energy consumption, and as a function of parameter growth, storage of data samples, or a pool of model copies. In particular, the TinyML community defines such requirements around a memory of 100KB to 1MB, and a processing power between 10MHz and 100MHz [144]. Moreover, to handle high-speed data streams, the model should work in a limited amount of time, ensuring it can incorporate new information as it arrives, predict, and update the model at any point.

Two communities. To tackle these challenges of bringing advanced AI to edge environments, two parallel research communities have emerged, with their own predefined data schemas and prediction tasks specifically tailored for on-device edge training. Both these communities recognize one cannot repeatedly access data from streams and reconstruct new models from scratch. Instead, they propose some form of stateful learning, detecting patterns without the need to store all the data and pass through them multiple times.

Neural networks. The first community focuses on deep learning algorithms, being more fragmented in its solutions for edge requirements, mostly identified by "online continual

*This work was funded by the EU, through the Portuguese Republic's Recovery and Resilience Plan, within the project PRODUTECH R3. It was also funded by the Portuguese Foundation for Science and Technology under project doi.org/10.54499/UIDP/00760/2020 and Ph.D. scholarship PRT/BD/154713/2023.

*Corresponding author
fonso@isep.ipp.pt (A. Lourenço)
ORCID(s): 0000-0002-3465-3419 (A. Lourenço); 0009-0008-8702-3876 (J. Rodrigo); 0000-0003-3357-1195 (J. Gama); 0000-0003-4417-8401 (G. Marreiros)

learning", "TinyML", "lightweight AI", or "task-free learning" [144, 140]. It mostly focuses on including mechanisms to shrink the size of NNs, such as pruning, and quantization, [59, 62], and incrementally processing mini-batches of data from varying distributions, precluding multiple epochs of offline training and the storage of data [71, 16]. The major challenge of these methods are catastrophic forgetting, and the very slow convergence rate in performing stochastic updates to these entangled model structures. Since no weights are fixed, previously learned knowledge can easily be overwritten when training on new data from a different distribution. Moreover, this plasticity does not necessarily imply that NNs can always learn new data efficiently. In fact, they become extremely data inefficient as more data is learned, requiring various passes of old data to mitigate the interference between old and new representations. Moreover, as deep learning architectures have been crafted with inductive biases for matching invariances and homogeneous data, they struggle to learn the irregular patterns of IoT tabular data [47].

Decision trees. The second community recognizes itself by keywords such as "data stream mining", "concept drift", and "online learning" [28, 89]. It mostly focuses on ensembles of resource-efficient tree-based learners that rely on statistical bounds of incrementally computed information gain measures to determine whether the observed utility of a split is statistically significant [44, 48]. The major challenge of these methods is a lack of plasticity. As the tree grows from the root node, the descendant nodes subsequently get fixed to covering particular sub-spaces of the parent node. Consequently, this community focuses on developing advanced methods to not only carefully select when to expand, but also to prune affected parent nodes, instead of a complete replacement of the base learner [29]. Overall, these shallow learners possess a faster online convergence rate, thanks to their simple model structures with a small number of trainable parameters. However, due to their limited learning capacity, they can end up with inferior performance than deep learners, whose low-dimensional hidden representations allow better feature interplay on complex raw inputs.

This survey. Despite this extensive research from both communities, a key gap exists in integrating these areas, particularly with respect to memory optimization and the handling of tabular data streams for on-device training. While continuous learning algorithms have made strides in addressing the challenges of catastrophic forgetting and learning from sequential data, they often do so without considering the unique constraints of memory-limited edge devices that process IoT data [140]. Conversely, efforts in reducing computational overhead through pruning or quantization, are often disconnected from the challenges of incrementally incorporating and preserving knowledge from heterogeneous, feature-rich, and dynamic nature of IoT data. To bridge these areas, this survey proposes to answer the following research questions, each one being respectively addressed by a section:

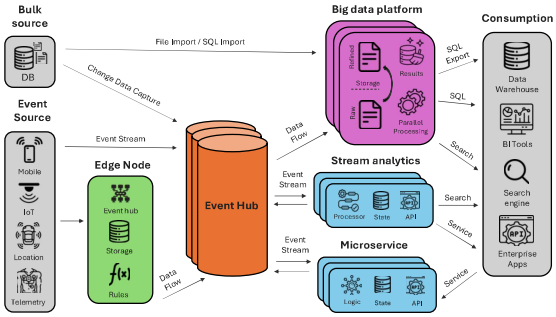


Figure 1: Data mining architecture

RQ1: What are the fundamental requirements for developing on-device training solutions? How do different data architectures (batch vs stream) and network capacities (cloud vs edge) influence algorithm design?

RQ2: How can NNs be deployed at the edge while continuously learning, under resource constraints, open-world settings, and IoT tabular data?

RQ3: How to make DTs viable for on-device training, in terms of dynamic architectures, memory-efficient growth, ensemble learning, and without supervision?

RQ4: How can the suitability of an on-device training strategy be assessed, and what evaluation metrics ensure robust model performance before deployment?

2. On-device edge learning

To address RQ1, this section aims at redefining the requirements for developing on-device training solutions. For this purpose, two major drivers of edge applications were discussed, along with consequence AI algorithm design implications. On one hand, how contrasting approaches to processing data, namely in batch or stream architectures, lead to different AI training paradigms, with an inference engine either trained offline or online. On the other hand, how different network capacities, caused by moving data processing closer or further from where the data is produced, lead to different AI resources paradigms in CloudML, MobileML and TinyML. These topics will refer to Figure 1.

2.1. Data architecture

In the IoT environment, several data processing architectures are employed, each designed to address the specific characteristics of the data. These include batch processing for static datasets, which is commonly implemented using Extract, Transform, and Load (ETL) pipelines and data lakes; batch processing for stream data, often facilitated through event hubs and publish-subscribe (pub-sub) communication; and stream processing architectures that can handle both batch and stream data, typically leveraging change data capture (CDC) techniques.

Big data platform. Batch architectures are most commonly use in applications that do not require immediate responses, but instead focus on algorithm performance and

understanding patterns within the data [145]. In a typical batch processing pipeline, data is handled through ETL operations [139]. Once transformed, the data is saved in transactional databases, such as MySQL, where it is structured into tables and then into schemas optimized for complex analytics queries [25]. To deal with the vast amounts of diverse, unstructured information, raw data can be alternatively stored in data lakes [145]. Typically, a batch system retrieves records on a scheduled basis, e.g. monthly, via APIs. Once the data request is fulfilled, all files are imported, the connection is halted, and the processing begins. Technologies such as MapReduce or Spark are typically used in the processing layer, after which results are stored and delivered to the final consumer [145]. However, while data is processed in bulk its collection should take into account the sequential nature of data, such as in engine readings. To address this issue, a data streaming processor can act as a mediator, allowing downstream systems to consume events at their own pace and in the correct order [80]. Unlike data lakes, where all services read from the same queue, thus creating complications due to the need for additional filtering, a data streaming processor organizes messages into queues, storing them as if they were in a long log file, with data being directed into separate streams or topics [75]. This approach models data as transient streams rather than persistent tables, enabling services to subscribe only to their specific topics of interest [145].

Event hub. For this purpose, a streaming processor uses the pub-sub communication method. In contrast to traditional web communication via APIs, where the system synchronously sends a request and waits for a response, pub-sub enables asynchronous communication between multiple systems, allowing large volumes of data to be generated simultaneously. This effectively decouples data sources from data consumers. Data is divided into topics or profiles, and consumers subscribe to these topics. When a new data record or event is generated, it is published within the topic, allowing subscribers to consume the data at their own pace. This method enables systems to handle thousands of events per second without the delays associated with synchronous responses [80]. However, data is still processed in batches at regular intervals. Thus, to alleviate the consequent batch processing bottleneck, data is buffered in an event hub. Ultimately, the issue of high latency remains. By the time files are obtained from the source, they are already outdated. Loading them into systems such as Hadoop, and subsequent processing steps, such as using MapReduce or Spark, further delay the data's freshness [75, 139]. As a result, the final output can not reflect up-to-date results, with IoT data losing value over time. While event hubs provide an effective buffering solution, the asynchronous nature of data storage and analytics necessitates innovative approaches to ensure timely data processing and retrieval [139].

Stream analytics and microservices. To address this, streaming architectures emerged, where analytics are performed on each event as it arrives, e.g. when detecting an anomaly, send an event back to the event stream for further processing or action by another service. For this purpose, streaming architectures typically consist of multiple services, each handling specific tasks, rather than a single monolithic system. This separation of concerns is crucial for scalability and maintainability. Microservices can consume data from event tables and produce new events back to the event stream, e.g. a dispatch service, using a ML model to analyze vehicle speed and location in real-time, can continuously send adjusted results on estimated arrival times. In such systems, dealing with limited event information to minimize message size is common. However, additional information might be required to enrich stream analytics [6]. For instance, in a scenario where vehicles are monitored in real time, the system might transmit data such as the vehicle ID, driver ID, vehicle position, and some driver behavior metrics. While this event data provides valuable real-time insights, more in-depth analytics may require supplementary information about the truck or driver. In such cases, instead of transmitting all of this data with each message, CDC techniques allows only the changes in the data to be captured, ensuring that the event hub remains up-to-date while minimizing system load [25]. With CDC in place, two streams are consumed: the event stream and the stream of event changes. These streams are joined using a stream-to-static Join, where dynamic event data, such as vehicle positions, is merged with static data, such as driver details. The static data functions as a cache, enabling real-time analysis without requiring redundant information to be transmitted with every event message [139]. Furthermore, to manage the varying data workloads of event data and historical data, streaming and batch processing are often combined, with historical data being stored in a big data platform or an object store, and an event table operating in a pub-sub model, where one subscriber processes the event stream in real-time, and another stores raw data for historical analysis. One example of this is the lambda architecture [80]. Alternatively, one can rely solely on stream processing, e.g. via a Kappa architecture [75].

2.2. AI learning

As a consequence of the diverse data architectures employed for processing, three distinct AI learning paradigms emerge: (1) offline inference with offline training, (2) online inference with offline training, and (3) online inference with online training.

Offline inference / offline training. The 1st paradigm is typically applied to data at rest, where algorithms process the data multiple times under the assumption that the data samples are drawn from a stationary probability distribution. This assumption allows models to use greedy, hill-climbing search strategies within the model space [75]. In practice, data scientists often rely on data warehouses for structured data and query data lakes for raw, unstructured data to support offline learning [145].

Online inference / offline training. In the 2nd paradigm, model metadata, hyperparameters, vocabulary, and learned weights are periodically analyzed offline and applied using a

static inference engine. It is important to note that while data at rest loses value when processed in bulk, data in motion can also lose value, as its patterns are extracted by outdated algorithms with low frequency retraining.

Online inference / online training. The 3rd paradigm addresses this issue by continuously updating the inference engine, eliminating the need for periodic retraining. If such a model could replicate the learning capacity of traditional batch methods, while continuously mining high-volume, open-ended data streams as they arrive [139], the extracted value from the data would be maximized. However, such guarantees are not feasible, and trade-offs are inevitable. Online training inevitably requires incorporating summaries of prior data, e.g. representative samples or decision models.

2.3. Network capacity

In the IoT environment, it is essential to consider not only the selected data processing architectures and learning algorithms but also their deployment locations, and the corresponding network capacity.

Edge nodes. For instance, remotely located servers connected via the Internet offer significant processing power for managing and analyzing large-scale IoT data. However, these servers can become costly and inefficient when tasked with handling real-time data ingestion at high volumes and strict response time requirements [75]. The bandwidth required to transmit data to and from the cloud can be substantial, and while predictions can be made online after models are trained offline, real-time processing at scale remains prohibitively expensive, especially in systems with numerous devices and high-speed data streams. Furthermore, high-frequency sampling coupled with bandwidth latency can lead to issues such as data arriving out of order, disrupting the intended sequence of events. To address the challenges of insufficient capacity for transmitting all data to a centralized data center, edge nodes can be deployed closer to the event source. While edge computing is not meant to replace cloud computing, it complements it by enhancing system efficiency, accelerating tasks typically handled by the cloud, and reducing both transmission and computation times.

2.4. AI resources

As a result of varying processing locations and hardware choices, the resources available for AI algorithms can differ significantly [125]. Machine learning algorithms can generally be categorized into three groups: TinyML algorithms, optimized for battery-operated devices with limited resources, typically around 100KB of RAM and low power consumption; MobileML algorithms, designed for devices with storage capacities up to 8GB, balancing accuracy and efficiency; and CloudML algorithms, which prioritize accuracy over computational constraints, often ignoring hardware limitations [144].

TinyML. Table 1 presents four examples of TinyML devices commonly used in IoT applications. As a general rule, devices with clock speeds below 100MHz typically have around 256KB of SRAM, while those with clock speeds above 100MHz tend to have between 256KB and 8MB of SRAM [125]. These devices are usually powered by coin or Li-Po batteries, which require power in the milliwatt range [144]. However, these hardware requirements are constantly evolving as technology advances.

Table 1
Examples of edge devices for TinyML: a) Himax WE-I Plus HX6537-A, b) Expressif EYE ESP32-D0WD, c) Arduino Nano 33 BLE nRF52840, d) SparkFun Edge ArtemisV1

	Clock	Flash/RAM	Sensors	Radio
a)	400MHz	2MB/2MB	Acc, Mic, Cam	None
b)	240MHz	4MB/520kB	Mic, Cam	WiFi
c)	64MHz	1MB/256kB	IMU, Temp, Gest, +	BLE
d)	48MHz	1MB/348kB	Acc, Mic, Cam	BLE

2.5. On-device learning requirements

Faced with bounded compute and time, an online autonomous system needs to make implicit and explicit trade-offs, both at a stability-plasticity and a forward-backward transfer level, where the minimum achievable error rate is given by the knowledge provided in the data and hardcoded in the model's inductive biases. Thus, being essential to consider if the assumed constraints are actually useful for the search space at hand. For instance, to design for more restrictive real-life situations, the notion of on-device training is usually associated with overly strong constraints, depicted in Figure 2, e.g. single pass learning with each instance at a time, and assuming no data storage. While the intention of these requirements may seem logical, defining whether an algorithm should fulfill each one of these constraints implies a trade-off that needs to be analyzed. For example, on-device training may consider incorporating high-frequency mini-batch updating techniques. Additionally, the decision of whether mini-batches are revisited multiple times by the algorithm presents another trade-off. Furthermore, experience replay approaches that keep a small amount of data storage space of previous knowledge, controlling data processing within small batches of instances via curriculum learning, or embedding knowledge on the training algorithm to reduce the plasticity as a bounded data stream nears its end, can all be highly beneficial for on-device training.

Natural data flow. If one must define the constraints of what on-device training is, then the concept of data arrival order and execution model becomes far more critical. On-device ML should operate within a pipelined execution framework allowing data to flow seamlessly through the stages. Regardless of the aforementioned strategies of mini-batching, saving a storage of samples, etc., the primary consideration is that the stages must operate concurrently, with upstream algorithm components actively pushing data downstream as it becomes available. While the aforementioned considerations should matter, this definition brings into focus the core constraint of on-device learning: the inherent lack of control over the arrival of data and corresponding ground truth labels. With data in its natural order, full of temporal, spatial, and causal dependence, and labels that

Table 2
Traditional vs. on-device learning

Aspect	Traditional	On-device
Size	Bounded	Unbounded
Processing	Batches	Instance-by-instance
Repetition	Multiple	Single pass
Data dist.	Stationary	Non-stationary
Speed	Slow	Fast
Storage	All	None
Labels	All	None
Arrival order	Controlled	Natural
Execution	Serial, pull data	Pipelined, push data

while correct at the time of request may be outdated or even incorrect by the time they become available. It is important, however, not to confuse the issue of processing data in its uncontrolled natural order with the challenge of dealing with non-stationarity. The concept of distribution drift is tied to the expressiveness-convergence trade-off of the model. For instance, while shallow learners may need to fully adapt their representation to incorporate what they interpret as new concepts, deep learners can integrate that same new knowledge with minimal interference. Nonetheless, as deep learners become overwhelmed by data complexity, their convergence rate diminishes, resulting in longer recovery times when adapting to new distributions in an already saturated representation. In contrast, shallow learners may exhibit faster recovery rates, but only if they can effectively represent new concepts [129]. Ultimately, it is not the data speed that matters, but the interplay between data arrival, training, recovery, and inference. The core challenge lies not in the nature of data itself, but in the computational approach, whether the information is processed immediately as it arrives at each stage, or after being accumulated and processed in earlier stages.

Objective evaluation. Most importantly, to avoid getting caught in the complexities of defining what on-device learning should look like in practice, solutions must be evaluated according to multi-criteria performance metrics tailored to specific edge applications. Intuitively, this can be done by monitoring the learning and retention ability, based on average accuracies and forgetting measures. Moreover, while these output-based metrics are valuable, it is equally important to assess the quality of the representation learned by each algorithm. On-device continual learning is more aptly described as a second-order problem of representation learning, emphasizing how well a model's internal representations support future learning. Therefore, the focus should not solely be on decision boundaries, but also on how effectively the learned representation facilitates the incorporation of new knowledge. Further details on these evaluation strategies are provided in Section 5.

3. Neural networks

To address RQ2, this section describes the building blocks needed for deploying NNs at the edge. Firstly, addressing how to embed in NNs the ability to continuously learn, while balancing the stability-plasticity and forward-backward transfer trade-offs. Secondly, covering methods used to squeeze NNs in resource-limited edge devices. Thirdly, adressing how to overcome dependence on external descriptions and heavy supervision in an open world setting. Fourthly, analyzing how well NNs perform on the most dominant IoT data modality, i.e. tabular data, and techniques to improve its performance.

3.1. Stability-plasticity solutions

This section covers methods to address catastrophic forgetting, namely controlling how model parameters change between concepts while ensuring independent representations for each concept, capturing common structure within various tasks with an aggregated state abstraction, or decomposing concepts into reusable modules with a compositional representation. Moreover, methods to keep the capacity to learn from new data, as the model accumulates knowledge, are also detailed.

Independent representations. Regarding control of how subsets of model parameters change between concepts, various approaches can be taken. For instance, parameter isolation methods consist in dynamically allocating new neuronal resources when acquiring new knowledge [121, 157]. While this strategy directly avoids the problem of catastrophic forgetting, it exacerbates the curse of input dimensionality and storage requirements. As the number of concepts seen grows, there is a larger space of past representations to sift through. Alternatively, one can enforce that different concepts use different components of a single model, typically using attention vectors which protect previous concepts by masking their important parameters and hold-out a subset of its parameters for future learning [127]. Moreover, some methods use iterative pruning to compact, pick and grow NNs [126] or initialize context-specific parameters with an auxiliary network [107]. Intuitively, important parameters should not have their values changed, while non-important parameters are left unconstrained. To create an elastic memory, one can use Fisher Information [71] or KL-Divergence [103] for weighting in a quadratic parameter difference regularizer of the parameter posterior to control the change of model parameters between two learning tasks. Moreover, multiple disjoint subspaces can be created both at the gradient or feature level. Gradient-based methods impose the parameter update direction of the new concept to be orthogonal to the gradient subspace of the old concepts, ensuring minimal interference. Feature-based methods require that the parameter update direction of the new task is orthogonal to the subspace spanned by the input features of the old concepts [20, 148].

Aggregated state abstraction. While these approaches mitigate catastrophic forgetting by freezing previous concept representations and minimizing representational overlap, they also minimize the potential for positive forward and backward transfer. Instead, one can tackle forgetting by preserving the input-output relations as the actual object of interest, disregarding how different can be the parameters that produce the same predictive behavior. For instance, reinforcing the importance of old concepts using experience replay, with four key components potentially tuned: (1) rehearsal representation; (2) label strategy; (3) rehearsal policy; and (4) buffer maintenance policy. For the (1) rehearsal representation, one can store raw input data and latent features from hidden layers on a memory buffer [54], e.g. by replaying compressed representations of activations of the intermediate layers in addition to the input-output pairs [16], or producing these synthetically with a generative model [131]. For the (2) label strategy, the memory buffer can hold the true label, which may not always be feasible in real-time applications, or a predicted label by the model. For instance, one can use additional logits distillation, referred to as dark experience replay [15]. Using these soft targets can be interpreted as functional regularization by generalizing the constraining metric to be a divergence on the output conditional distribution [19]. In contrast to the aforementioned parameter regularization, no assumptions are made on the parametric form of the parameter posterior distribution, which allows models to flexibly adapt their internal representation as needed. For the (3) rehearsal policy, one can identify which stored samples should be selected for rehearsal using a variety of sampling policies, e.g., uniform balanced, min rehearsal, max loss, min margin, min logit-distance, and min confidence [18, 112], or even making virtual updates with the incoming data to find the maximally interfered old samples for a loss-based policy [3]. For the (4) buffer maintenance policy, most techniques use reservoir sampling, where a new sample overwrites a randomly selected sample from the buffer once the buffer is full. However, since the reservoir buffer population approximately follows the data distribution, it can severely deteriorate the performance of underrepresented domains. To alleviate this, one can use a class-based reservoir scheme [23], exploit the statistics of the stored samples [112], or using classification uncertainty and data augmentation [10].

Compositional representation. While these methods allow to learn a shared representation, such effort has proven to be equivalent to learning each new concept in a constrained parameter space and NP-hard in general, as the feasible parameter space tends to be narrow and irregular as more concepts are introduced [73]. In fact, these non-modular methods fail to capture the intuition that, in order for knowledge to be maximally reusable, it must capture a self-contained unit that can be composed with similar pieces of knowledge. To alleviate this issue, modular architectures with compositional knowledge allow solving combinatorial previous concepts to learn new ones. There are different types of modularity: data-based, which can be both intrinsic or imposed; concept-based, which relies on sequential and parallel sub-task decomposition; and model-based, which relies on static and conditional composition of sequential or non-sequential modules [136]. In practice, modularity is often achieved with dynamic networks, e.g., designed via neural architecture search [82], presenting dynamic depth, width, and routing. For instance, multiple network branches can be parallel built as experts, whose outputs are selectively executed with a gating module and fused with a data-dependent weighting module [130]. However, such solution doesn't fully eliminate weight sharing across modules and instead only gates the sharing. Conversely, routing-level solutions allow for higher specialization, decomposing the pool of sub-concepts in different processing stages or time steps, not only in parallel but also in sequence. Indeed, because of this, modules can be used at multiple locations in the network, and the algorithm can better learn to share parameters dynamically using a composing strategy to choose different parametrized functions depending on the context. Given an input the router makes a routing decision, choosing a function to apply and passing the output back to the router recursively, terminating when a fixed recursion depth is reached [106]. Because routing networks jointly train their modules and the router, their major challenge consists in stabilizing the interaction of heterogeneous modules and diverse choices made by the composition strategy training [72]. In conditional computation, when there is a premature selection of a few modules, which are trained more rapidly, a self-reinforcing effect leads to missing diversity and causes modules collapse [119]. To counter this effect, one can regularize module usage towards a desired form of a sub-optimal minimum [130]. Moreover, this problem extends to the fact that training a routing network is non-stationary from both the perspective of the router, and of the modules, as they mutually depend on each other. Consequently, most methods either fix the composition strategy while learning the modules; or fix the modules while learning the composition strategy. But in its most general form, the compositionality problem should be to jointly learn both [119].

Capacity to learn. Despite the aforementioned methods tackling the key issue of catastrophic forgetting for on-device training, these didn't consider that, as the model accumulates knowledge, capacity to learn from new data decays. In such situations, one might not want to discard any irrelevant information that is still retained in the model's memory. Naturally, one could simply use larger networks which tend to maintain plasticity longer in face of new concepts [121, 157], however such strategy wasn't designed for a setting of finite memory in edge environments. Instead, plasticity injection lies in a minimalist intervention that increases the network plasticity without changing the number of trainable parameters, by freezing the current model and create a new one which learns the change to the predictions [104]. Moreover, capacity to learn new data is intrinsically related to maintaining plasticity [91]. As networks converge to the current concept's local optima, they depart from their initial random distribution, losing the ability to adapt to new

learning signals, especially after training for long sequences of concepts [27]. Thus, instead of reactively injecting capacity, one can preventively maintain the plasticity properties of the initial random distribution of network weights via diverse non-saturated units and small weight magnitudes [2]. Furthermore, regularization techniques can greatly influence the optimization landscape. For this purpose, instead of regularizing towards zero which is likely to collapse the ranks of the weight matrices, one can regularize toward the initial parameter values [76]. Thus, maintaining smaller weight magnitudes, avoiding saturated activations, and preventing weight rank from collapsing. Alternatively, one can allow the parameters to deviate further from initialization, by instead preserving the curvature properties for desirable plasticity, taking the difference of order statistics using a Wasserstein regularizer [78]. These techniques are similar to the aforementioned constraining of parameters under the umbrella of catastrophic forgetting [71]. However, instead of regularizing towards the parameters of a past concept to preserve old knowledge, these regularize towards initial parameters to remember how to learn. Furthermore, these strategies focus on unit-level interventions, thus potentially missing out on global properties of the loss landscape. To address this, one can regularize a feature subspace to its initial value [91]. Moreover, one can also shrink all the weights and inject randomness, in order to stop the weight magnitude from continually increasing and reduce the percentage of saturated units [8]. However, this approach can be somewhat drastic, causing the model to forget previously learned knowledge. To alleviate this, one can guide this process, by resetting units which have been saturated [133], or using more sophisticated approaches. For example, tracking a mean-corrected contribution utility of the outgoing weights and a adaptation utility of incoming weights for each neuron [27], in order to reset a neuron when the product of these utility measures goes below a certain threshold, while also protecting new units from immediate re-initialization with a maturity threshold. Overall, resetting improves plasticity, but it can slow down convergence on single concepts, while still not resolving the fundamental signal propagation issues of stable representations [91].

3.2. Memory constraints

Most of these models employ multiple memory systems without considering processing, memory, and time limitations, which severely limits their deployment on resource-limited devices and real-time applications. To circumvent this issue, numerous model compression and acceleration methods have been proposed, both at a data and algorithm level.

Algorithm-level. From the algorithm side, common techniques are: pruning, which lies in removing less significant weights and connections [51, 85]; quantization, which reduces the precision of the network's weights and activations [22, 102]; lightweight NN designs, like MobileNets [59] and ShuffleNet [163], whose architectures are specifically crafted to be efficient from the ground up; low-rank factorization which further enhances efficiency by decomposing weight matrices into lower-dimensional forms [64, 26]; and self-distillation in which a NN uses its own predictions to iteratively refine and improve its performance [160, 161]. Regarding pruning, one can employ L0 or L1 regularization to induce sparsity directly [84, 87], or identify the connections to prune with a inferred measure, such as the L2 weight magnitude [51, 109], first-order [100] and second-order [30] information, or both [99]. However, most of these methods focus on removing individual weights from the network, resulting in highly irregular sparse models that require specialized hardware and software implementations for efficient execution. To circumvent this issue, one can create faster models by removing entire structures within a network, such as channels, filters, or layers, thus maintaining compatibility with existing hardware accelerators and software libraries [146, 79].

Data-level. From the data side, one can try to efficiently condense more information into the memory by sequentially distilling the knowledge from the data, while keeping the model fixed. Thus, these methods optimize the pixel values of distilled images, instead of optimizing the network weights for a particular training objective. For instance, it has been shown that it is possible to compress 60,000 MNIST training images into just 10 synthetic distilled images (one per class) and achieve close to original performance with only a few gradient descent steps, given a fixed network initialization [141]. In fact, these small number of data points, given to the learning algorithm as training data, do not need to come from the correct data distribution in order approximate the model trained on the original data.

3.3. Supervision assumptions

Most of these models depend on external descriptions and heavy supervision assumptions, failing when there are unknowns, referred to as the open world setting [124]. For instance, compositional representation methods without access to the information about which part of the input should be used for module selection would perform poorly for samples that are compositionally out of the training distribution. To circumvent this issue, most of these models assume that semantic information is explicitly given in the form of external task descriptors, attribute information, pretrained modules, or hard-coded modular structures [124, 70]. In classification tasks where semantic information is available, it is only necessary to consider known known classes (KKCs) and known unknown classes (KUCs), i.e. positive and negative training samples, respectively. However, in evolving context-dependent scenarios, additional categories must be considered. These include unknown known classes (UKCs), which have limited training samples but are associated with accessible side-information, and unknown unknown classes (UUCs), which lack both training samples and semantic side-information.

UKC. To tackle a UKC, both generalized few shot learning and zero-shot learning are used. Generalized few-shot learning leverages the relationships between KKCs and a

small set of labeled samples resembling the UKC, incorporating shared semantic information across KKCs, UKCs, and the available labeled samples [142]. In contrast, generalized zero-shot learning relies exclusively on semantic information shared between KKCs and UKCs to recognize novel categories during testing. Here, test samples originate from both KKCs and UKCs, but no labeled instances similar to the UKC in question are provided during training [151].

UUC. However, total autonomy requires models to learn exclusively from the implicit structure inherent to the input data, leveraging internal cues to distinguish between concepts [142]. Under this paradigm, UUCs can be addressed through open-set recognition and novelty detection methods. Open-set recognition handles UUCs by rejecting them when only KKCs are available, without additional side-information, such as attributes, and when there is a limited number of UUC samples [43]. Conversely, it can also recognize UUCs when semantic information shared between KKCs and UUCs is available. Novelty detection methods, on the other hand, address UUCs without relying on any explicit side-information by decomposing the problem into two levels: (1) distinguishing between learned and novel concepts, and (2) incrementally learning and solving each individual concept. For instance, performing cross-concept class discrimination [49] or identifying concepts via predictive and parameter uncertainty, e.g. using conditioned hypernetworks [107], or learned binary masks [148]. Moreover, one can explicitly model the cross-concept identification, e.g., using a separate network [1], a differentiable Bayesian changepoint detection scheme [53], or any out-of-distribution detection method [70].

3.4. Tabular data adaptations

Learning from tabular data forms the backbone of numerous real-world IoT applications, largely due to the wide adoption of relational databases in these domains. NNs have enabled remarkable advancements in handling unstructured data, such as text and images. However, when applied to tabular data, NNs often underperform DT ensembles [14].

Tabular challenges. As deep learning architectures have been crafted to create inductive biases for homogeneous data, matching invariances and spatial dependencies of the data, they struggle to learn irregular patterns of the target function, with their rotation invariance hurting performance [47]. While in text and vision, data is intrinsically tied to the position of the word/token or pixel, the order-invariance of tabular data hinders position-based methodologies, e.g. convolutions, less applicable for tabular data modeling. This lack of spatial or temporal structure that could inform a good prior of the data makes it challenging for the model to understand the inherent relationships between features. Moreover, this is further aggravated when handling heterogeneous and uninformative features, skewed heavy-tailed feature distributions, and extreme values [47]. NNs trained on tabular data tend to develop overly simple decision boundaries, biased to overly smooth solutions and low-frequency functions. In contrast to extensive preprocessing required for NNs, DTs handle these issues effectively by approximating missing values, using appropriate thresholds and managing variable ranges internally during the splitting process [132]. To circumvent these issues, various techniques have been proposed to make deep learning work on tabular data, namely through data transformations, regularization models, differential trees and transformer-based models [14].

Data transformations. Firstly, one can convert the tabular input into a homogeneous data format, e.g. by transforming tabular data into images for direct application in a convolutional NN [138]. Moreover, for numerical features, embeddings have shown to provide powerful initial representations, e.g. replacing the original values with bins indices constructed with quantiles of the individual feature distributions or the target values, using Johnson distribution or piecewise linear transformations [45]. Alternatively, one can use more complicated automatic techniques that are part of the model architecture, e.g. using a contextual embedding with self-supervised learning [158].

Regularization models. Alternatively, one can model the varying importance of tabular features and the multiplicative interactions between them. For instance, by using gradient attributions to encourage neuron specialization and orthogonalization [65], a counterfactual loss [67], an asymmetric encoder-decoder self-supervised framework [150], constructing and projecting into a prototype-based space [156], or stochastic gates [153].

Differential trees. To benefit from the inductive biases of tree-based methods, these can be extended with a smoothed decision function. For instance, by combining gradient-based optimization with hierarchical representation learning [111], using a gradient boosting framework with shallow NNs as weak learners [9], using tree-inspired sequential attention for feature selection [7] or distilling a NN into a soft DT [35].

Transformers. Alternatively, one can use token-based mechanisms for feature selection and reasoning in heterogeneous tabular data [132]. For instance, combining column descriptions and cells as input for feature encoding [143], row and column attention mechanisms to capture the inter-sample interactions with self-supervised pretraining to deal with label scarcity [134], additive attention to consider the interaction between each token and the global representation, achieving a linear complexity [149], transforming categorical features into contextual embeddings [60], or sequential hierarchical subnetworks prioritizing the most significant features [7]. Furthermore, some of the aforementioned tabular transformers [149, 134], when pre-trained with a diverse range of tabular datasets, have been shown to exhibit better performance via finetuning than training from scratch [164]. Following these findings, various large tabular models pretrained on synthetic tabular data have been proposed for classification, which yield great promise for application on edge applications [57, 13]. By using on-the-fly sketches to summarize unbounded streaming data, one can feed this information into the pre-trained foundational model for efficient processing [88].

4. Decision trees

To address RQ3, this section describes the adaptations needed to make DTs viable for on-device training at the edge. Firstly, addressing how to dynamically adjust the tree structure to new data, via approximation-based splitting. Secondly, describing strategies that allow to control the tree growth efficiency in memory-scarce edge environments. Thirdly, dynamic combinations thereof in ensembles for better performance, including meta-learning and model-based clustering techniques to model a concept history. Fourthly, how to overcome supervision assumptions. In reviewing these methods, it is important to note that while NNs can equally evolve via both dynamic architectures, i.e. adding new parameters, and fixed architectures, i.e. activating the relevant part of the network without changing the architecture, the algorithm properties of DTs put more emphasis on adaptation via dynamic architectures, i.e. structural expansion and pruning.

4.1. Incremental learning

To incrementally construct a DT at edge devices, using contained memory and time per sample, the fundamental algorithm design component is approximation-based splitting [29]. As new instances arrive, they traverse the tree from the root to a leaf node, updating statistics at each node that guarantee for split decisions almost identical to those that conventional batch learning methods would make, despite lookahead and stability issues.

Split decision. Based on these sufficient statistics, these DTs continuously update the heuristic values for each attribute, such as the traditional information gain (IG) and Gini index (GI) [28], fuzzy entropy [31] or a fair-enhancing gain [162]. With this, the decision to split is built upon the concept of a statistical Bound which quantifies the confidence interval for the heuristic function, based on a minimum amount of data needed to make a statistically reliable decision when the data distribution is unknown. For instance, comparing the Hoeffding Bound (HB) to the difference of heuristic obtained between the best attribute split and second-best attribute [28]. Once the difference in evaluation surpasses the bound, the leaf node is split into child nodes. However, this HB can be too restrictive and its expected guarantees not given. Firstly, it only applies to numerical averages, however both IG and GI cannot be expressed as arithmetic average. Secondly, it demands that these objects are independent variables, with the observations used for the computation of the split criterion being statistically independent. In most cases, the assumption of independence is not valid, primarily due to the overlapping nature of sliding windows and the inherent sequential characteristics of data streams. Alternatively, one can directly use the classification error as a splitting criterion, rather than a concave approximation of it, like the entropy or the Gini index [122], decompose the entropy gain calculation into three components and applied the HB to each one of them [32], or use the McDiarmid's bound, which provides tighter guarantees in situations where the impact of changing one variable is limited by a known constant, being tailored for functions of independent random variables with bounded differences [123, 24]. As for the leaf classification strategy, despite most algorithms using majority voting (MV), VFDTc has shown that one can apply the naturally incremental naive Bayes (NB) classification in tree leaves for better exploitation of the available information [39]. While VFDT only uses a crude approximation about class distributions, NB takes into account not only the prior distribution of the classes, but also the conditional probabilities of the attribute-values given the class. Alternatively, one can also dynamically switch between MV and NB, depending on the amount of data available.

Split point. Another challenge is finding the split value. For categorical attributes, memory requirements are relatively low because the algorithm only needs to store the frequency counts of each unique value [28]. For continuous attributes, however, it is impossible to evaluate all possible split points to determine the optimal choice, and on-device training prohibits knowing the range of values a priori, thus inhibiting proper discretization [38]. To address this issue, the original VFDT used a binning approach where for every new unique value observed a new bin is inserted into the range. A single set of bins is shared by all of the classes. Once a predefined limit of 1000 bins is reached, the bins remain stationary, but their counts are incremented. Thus, not only being influenced by just the first 1000 unique values, but also requiring a lot of processing time and memory usage [28]. VDFTc mantains a binary search tree at each leaf node, dynamically inserting new values, and tracking class counts on either side of each cut point [39]. Alternatively, the distribution of values can be approximated by a single Gaussian [36], with the range between the minimum and maximum observed values being divided equally into n parts. For each of these split possibilities, the distribution of classes to the left and right is estimated based on the area under the normal distributions [58]. Thus, requiring the update of a few counts per class. Moreover, it has been shown analytically that by using Gaussian distribution [58] as sufficient statistics and misclassification error [122] as impurity measure, the best split value is where two Gaussian distributions intersect each other [96].

4.2. Dynamic pruning

The aforementioned choices allow to produce increasingly larger trees that can divide the problem space into finer regions, better approximating the target concept. However, such tree growth still remains highly uncontrollable, which can lead to two main issues: excessive memory consumption due to multiple redundant splits on all features, and a loss of plasticity due to the descendant nodes getting subsequently fixed to the space covered by their parent node. To address this, various pre-pruning and post-pruning techniques have been proposed. Pre-pruning strategies include splitting rules enhancements, adaptive tie breaking, adaptive grace periods, activity-based node decisions, and removing poor attributes. Post-pruning includes various methods that estimate in each

decision node whether the corresponding rooted subtree is consistent with the current data, then performing some action to reconstruct the subtree if an inconsistency is estimated.

Split rules. Traditional incremental DTs primarily evaluate the top two attributes for splitting. However, additional splitting criteria can be introduced to enhance the quality of splits. For example, constraints can be implemented to compare current metrics with both historical and cross-leaf data, while a skipping mechanism can be used to bypass strict splitting conditions when significant data changes are detected [21]. Another approach involves monitoring the fluctuation of the HB by tracking its mean, minimum, and maximum values, along with accuracy metrics, to serve as pre-pruning conditions for splits [155]. Additionally, a constraint can be incorporated to assess whether the top-ranked splitting feature at a leaf node provides substantial information gain compared to previous splits along the same branch [11].

Tie breaking. When two attributes have statistically similar IG, tree expansion can stall, even if either attribute would be a viable splitting choice. To avoid this, a split is triggered when the bound falls below a predefined threshold, ensuring a minimum growth rate. However, this fixed threshold can lead to premature splits, not due to a meaningful tie, but because of a lack of strong candidates. To address this limitation, an adaptive approach can be used, where a threshold is dynamically adjusted based on the mean of the HB, leveraging its proportionality to the number of input stream samples [154]. Another alternative is to determine tie-breaking conditions by comparing the difference between the top two candidates with the difference between the second-best and worst candidates, providing a more context-aware decision-making process [58].

Split attempts. To prevent premature splits due to insufficient sample sizes, which can compromise the reliability of the HB, decision trees impose a grace period, requiring a minimum number of observed instances before a split is considered. However, a fixed grace period that does not adapt to the data being processed can lead to inefficient computations, either by triggering unnecessary split attempts or by introducing delays in tree growth. Instead, a more effective approach is to adjust the waiting period dynamically based on observed tie-breaking patterns [58]. For instance, being adjusted dynamically after an unsuccessful split attempt, ensuring that the next iteration leads to a valid split. If the top-ranked attributes have somewhat distinct IG values, but not enough to justify a split, the algorithm can increase the grace period so that more samples are gathered until the HB falls below the IG difference. Conversely, if the best attributes are highly similar in IG but do not surpass the tie-breaking threshold, additional instances are required to reduce the HB below the threshold, with the grace period adjusted accordingly [42]. Alternatively, one can analyze class distributions from past split attempts to estimate the number of additional instances required for the HB condition to be satisfied [86]. To bypass this split attempt problem, a further optimization involves reducing the computational cost of split evaluations by replacing periodic heuristic recalculations with an incrementally updated heuristic measure, which can be maintained in constant time [137].

Leaf activity. Another pre-pruning strategy focuses on regulating split decisions based on leaf activity. The likelihood of selecting a suboptimal split depends not only on the number of examples within a leaf but also on factors such as feature space dimensionality, leaf depth, and the total number of instances processed by the algorithm [24]. One way to address this is by comparing the number of samples that reach existing leaves to those that do not [155]. If most samples continue to fall into existing leaves, tree updates can be delayed to prevent unnecessary structural modifications. Alternatively, leaves with low utility can be deactivated based on their probability of receiving new examples and their observed error rate [28]. When a better split is identified, the previous split's leaves are deactivated, and their statistics are stored. If, during a later evaluation, a deactivated split is again determined to be optimal, its saved statistics can be restored, avoiding the need to restart the process. A more adaptive approach normalizes leaf activity by measuring how many instances a leaf has processed relative to the average across all leaves since its creation [41]. Depending on this value, different strategies can be applied: inactive nodes can be halted to conserve resources, while high-activity nodes can trigger more aggressive expansion techniques, such as skipping constraints in the splitting process [89] or switching to a more flexible splitting rule [41]. These adaptive mechanisms ensure that computational resources are focused on expanding the most relevant nodes while preventing unnecessary splits in less significant areas of the tree.

Attribute removal. Finally, one can selectively discard leaf statistics for attributes that are likely to not cause a split. For instance, by evaluating after an unsuccessful split attempt, if the attribute's IG is less than the gain of the current best attribute by more than the HB [28]. To further reduce computational cost, one can introduce a dynamic candidate attribute set which keeps track of the top K attributes that are most likely to split. Only the heuristic measures of attributes in the set are updated continuously [137]. Also, one can complement both strategies, eliminating poor attributes that increase computational complexity, while grouping candidate good attributes to be used in the next iteration, instead of using all of the attributes [77]. Another perspective on this issue is to prevent from using features that have not been used thus far, by penalizing those that have similar gains to other features in previous splits [11]. Naturally, while the aforementioned pre-pruning methods allows to contain the model size, they are still unbounded. To address this, any of the aforementioned strategies can be guided by memory limits [28]. To avoid the costly check of the current memory, one can perform the precise calculation periodically, while estimating the size approximately whenever new nodes are introduced to the tree.

Split re-evaluation. Regarding post-pruning, methods can be characterized by two key steps: estimating in each decision node whether or not the subtree rooted at this node is consistent with the current data, and performing some action to reconstruct that subtree if inconsistent. To estimate consistency, one can re-evaluate every split in the tree to ensure that the current split remains the optimal one, e.g. checking if the current split outperforms the best alternative [61] or the null attribute [92]. Alternatively, one can directly monitor the error rate in nodes [63]. For instance, computing the error top-down with the prediction from the current split node [36]. Or, instead, computing it bottom-up with the prediction from leaf nodes [12, 86]. For this purpose, most approaches monitor streaming bits of misclassification [37, 12]. However, one can also monitor performance as a stream of real values, by mantaining a sliding window with relevant instances at each leaf node [105]. Both selectively forgetting outdated instances when leaves are in an improvement state, and aggressively shrinking window size when a persistent degradation of the performance measure is estimated [105].

Tree reconstruction. To reconstruct the tree, one can simply require another attribute to become the top split attribute [92]. If an improved split is detected, a new leaf node replaces the entire subtree rooted at the faulty node. However, while this split revision procedure can significantly reduce variance by simplifying the model, it may introduce considerable bias if important information is lost when the subtree is pruned. To compensate this, one can make new branches more readily, when it has evidence that a specific branch is better than none, i.e. a split with greater-than-zero IG with high probability, rather than better than any potential alternative [92]. In other words, instead of finding a split attribute that will never have to be replaced, the model aims at finding the current best available split attribute at a node until a better one is found. Alternatively, one can overcome this by instead restructuring the otherwise pruned subtree, leveraging on the fact that the order in which an instance traverses the nodes of a DT from root to leaf does not affect prediction [56]. Also, one can hold a short-term memory to train an alternative subtree that could quickly replace the corresponding original one [37, 105]. However, this strategy is only effective when the underlying change is abrupt, as the old classifier is immediately discarded. In real situations, changes are often gradual and consecutive concepts are similar. Therefore, an old subtree trained with many examples can be more accurate than a new subtree trained with a few, even when the performance of the old subtree has dropped significantly. To be more conservative, it is necessary to determine whether the alternative subtree is more coherent with respect to the current concept, and subsequently decide when to eliminate the alternative subtree that is no longer effective. For instance, building an alternative subtree as soon as an inconsistent node is detected, which only eventually replaces the original subtree, after a user-defined number of examples, when its accuracy is superior to the original subtree [61]. This alternate tree reduces bias while keeping the model relevant, ensuring that the new subtree adapts to the new concept. Moreover, one can perform this adaptation recursively in each alternative subtree affected [86], and even create new alternative subtrees in a split node regardless of a previous alternative subtree still being induced by a previous drift signal [12]. To leverage on the existence of these various alternatives, one can combine their predictions with a weighted vote approach inversely proportional to the error rates calculated by the change detectors [86].

Change detection. These tree reconstruction strategies impose two key things to monitor. Firstly, the prediction errors between subtrees, e.g. using fading factors with the Q statistic [40] in a user predetermined evaluation interval [63], or with a predictive sequential approach [86]. Secondly, the error of the trees susceptible to be pruned, e.g. using the Page–Hinckley [101] which is designed to detect abrupt changes in the average of a Gaussian signal [63], the adwin change detector [12] which provides mathematical guarantees for the false positive and false negative, or the hddmA test [34], which not only provides the guarantees, but also has constant time and space complexity [86].

4.3. Concept history

While single DTs offer simplicity, when memory allows, ensemble strategies can be another effective approach. Upon a concept drift, the diversity among trees can be exploited as some local minima are less affected than others. Affected components can be dynamically deleted, added, and combined with a voting scheme, weighting different weak learners based on their past performance. For instance, incorporating incremental versions of random forests [44], bagging [108] and boosting [48].

Pool. However, such ensembles constantly train all active base classifiers, progressively discarding some of them when a drift is detected. Base learners evolve and the previously learned concepts are forgotten before they reoccur. Consequently, these models might need to be trained from scratch, which results in a waste of computational resources, longer training times, and significant prediction errors while models are not up-to-date with the latest state of the data stream. To circumvent this issue, meta-learning [5] and model-based clustering [152] allow to more explicitly retain a pool of previous active and inactive concepts. This feature enables storing cold copies of previously learned base classifiers to be reused if they become relevant again in the future.

Recognition. Consequently, maintaining a concept history creates the need of identifying what concept is present at each time. For this purpose, there are 2 major categories of techniques acting as a trigger: conceptual equivalence and conceptual similarity. The former assumes that two classifiers describe the same concept when predicting similarly during a time window [5], while being enriched with automatic ensemble size calculations [110], and time dependencies modeled according to the concept drift properties [4]. The latter recognizes similar concepts based on cohesion and separability in the input space, e.g. euclidean distances between clusters representing different concept clusters [152]. In these, micro-clusters or latent features are

used to make a synopsis of the incoming instances and reduce the computational cost of finding similarities among conceptual vectors [66]. Then, methods differ in how they manage this concept history.

4.4. Lack of supervision

Across these methods, adaptation is mostly achieved via monitoring the standardized absolute error of model predictions, leveraging on the fact that changes in the decision boundary manifest in the error functions of learning algorithms as changing optima [101].

Unsupervised. To overcome the reliance on supervision, novelty detection methods use a concept history framework that learns the characteristics of normal patterns, enabling them to proactively address novel patterns in an unsupervised, dynamic environment where concepts may evolve either gradually or abruptly [33]. These methods employ clustering-based algorithms to distinguish sparse, noisy examples from distinct, cohesive concept types, by forming closed boundaries around normal patterns. The underlying assumption is that samples close in the input or latent space originate from the same concept. Evolving groupings are identified when sufficient cohesion is present within clusters, with separability allowing for differentiation between new groupings and established concepts or classes [33, 135].

Semi-supervised. However, while these learning algorithms have the advantage of requiring only unlabeled data, it is important to maintain the ability to leverage any available labels to better define decision boundaries. For instance, it has been shown that semi-supervised anomaly detection methods can outperform unsupervised methods even with only 1% of anomalous instances [50]. In real-world applications, it is easy to imagine a scenario where at least a few labels are available in deployment. Moreover, at the beginning of the learning process, models usually have access to a lot of labeled data. In these situations, one can couple the state-of-the-art performance of supervised learning algorithms [44, 48] with unsupervised mechanisms used to update the model during execution. For instance, using an ensemble of classifiers to maintain a knowledge base of previous concepts, where instances falling inside the decision boundary of the ensemble are classified by taking the majority of the votes in the ensemble, while the others are queued in a buffer, clustered using k-means and periodically tested for cohesion and separability [94]. Alternatively, using active learning mechanisms, e.g. based on instances being an outlier or the proportion of majority votes [93], while triggering label requests based on drifting classifier's confidence scores [52].

Change detection. Indeed, instead of using solely loss-based drift detectors to update state-of-the-art supervised learning algorithms [44, 48], one can extend these to deal with scarcely labeled streams. For instance, making the classifier more robust to extreme verification latency via a double-stage detector, which applies CUSUM for change detection and, then, silhouette detection for cluster validation of new classes [116]. Another option is to apply statistical tests on distributional changes in the input space, for instance, by using specialized committees of incremental Kolmogorov-Smirnov tests [118], or a discriminative classifier [46]. However, as these data-based drift detectors totally exclude the classifier from the detection process, this results in increased sensitivity to change and a large number of generated false alarms. In the extreme, if the underlying marginal data distribution $P(X)$ over the input does not contain information about changes in $P(Y \mid X)$, then it is impossible to exploit unlabeled data to improve a supervised learner or detect a change [165]. To circumvent these issues, rather than solely monitoring drifts through statistical tests on the data distribution $P(X)$, emerging drift detection methods focus on monitoring proxies of the conditional classifier probability $P(Y \mid X)$ in an unsupervised way. Examples include comparing the distribution of classifier output confidences [81], or tracking the proportion of samples within the margin of a linear support vector machine [128].

5. Evaluation

To address RQ4, one must firstly assess the suitability of the on-device training strategy according to the available relevant data, and hardware investments, relying on the notions discussed in Section 2 and other criteria, such as privacy and security needs. For instance, distinguishing between fast streams, e.g. in industrial machinery, which require immediate processing or even discarding instances to avoid potential data overload, and slow streams, e.g. in medical applications, which allow for more thorough data processing, and temporarily storing data for later use during periods of lower activity.

Representation vs data. After properly formulating the problem, one can focus on accurate multi-objective evaluation to study how different algorithms perform before deployment, both at a representational- and data-level [17]. These metrics not only assist in the development of algorithms at a more fundamental level, but also in the creation of better benchmarks, and the a priori estimation of the cost and limits of running learning algorithms in specific edge applications. From the representational-side, one can explore the value of hidden layers, e.g. by retraining the network while freezing layers, resetting blocks of layers to initial values, and re-randomizing parameters [159]. From the data-side, one can interpolate similarity in the data with linear combinations of pairs of instances from different concepts to probe continual learning performance effects [114], or study how hardness, complexity, heterogeneity and dissimilarity of the concept sequence can influence learning over time [68]. In this section, we bring attention to six different groups of techniques, illustrated in Figure 2: standard evaluation on unseen drifting data, parameter counting or norm calculations, flatness and robustness analysis, probing, representational similarity, and classifier comparison.

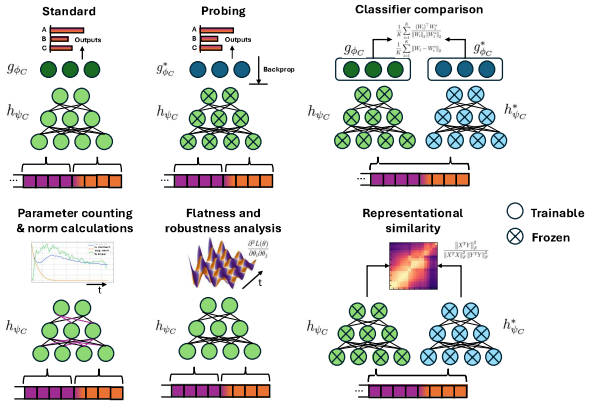


Figure 2: Methods of evaluation: standard, parameter counting or norm calculations, flatness and robustness analysis, probing, representational similarity, and classifier comparison

Standard evaluation. Common metrics are the accuracy, learning, and forgetting measures [18] using a per-iteration scheme with a window of size w. For instance, defining the windowed plasticity as the maximal accuracy increase in a iterations window, and the stability-based windowed forgetting as the maximal accuracy decrease. By varying this window length, one can account for forgetting and remembering capabilities at different time scales. Nevertheless, despite useful, these metrics don't provide a view on the maximum achievable performance. For this purpose, one can define the performance in comparison to baseline classifiers. One approach is to consider a model trained on all concepts jointly rather than sequentially [17]. However, this doesn't isolate forgetting effects, since concurrent training can improve the representation for each concept itself by contrasting a single class with a wider variety of other classes. Instead, one can use an ensemble strategy where training remains concept-per-concept and a model copy is stored after every concept [55]. This allows to reliably observe forgetting effects since the concept's original representation remains intact and they can all be simultaneously recovered by a concatenated head. Naturally, these upper bound baselines either require the whole data upfront or linearly more compute with every concept. For drift-related performance metrics [40], one can also consider: the mean time between false alarms (MTFA); the false alarm rate defined as 1/MTFA; the missed detection rate (MDR), the mean time to detection (MTD), and the average run length (ARL_θ) which generalizes over MTFA and MTD to quantify how long to wait before a change detection of size θ in the performance. For this purpose the following convention is used: before true change happens, all the alarms are considered as false alarms; after a true change occurs, the first detection that is flagged is considered as the true alarm; after that and before a new true change occurs, the consequent detections are considered as false alarms; if no detection is flagged between two true changes, then it is considered a missed detection.

Parameter count and norm. Regarding parameter counting, the simplest explanation for loss of plasticity is that the gradient norm goes to zero, eliminating all the network's expressivity. In practice, one can monitor the number of dormant units, i.e. neurons with ReLU's output being zero or sigmoid's and tanh's outputs being too close to either of the extreme values of the activation function [133]. Thus, not propagating gradients back to its incoming weights and severely slowing down learning. As for norm calculations, a steady growth of the network's average weight magnitude has been associated with plasticity loss, suggesting a slow divergence in the training dynamics, exploding gradients and numerical overflow errors [97]. Consequently, this can have a saturating effect on model components, e.g. softmax attention heads [83] and normalization layers [95]. Thus, resulting, for a fixed update step size, in smaller output changes than those obtained at initialization. Moreover, a more subtle issue can also arise when the norms grow at different rates across layers, leading to learning instabilities, due to updates to each layer having differential effects on the network output.

Flatness and robustness. While simple to compute, parameter counting and norm calculations can be too coarse in studying both sensitivity and generalization error of changing parameters [90]. Alternatively, flatness and robustness analysis of trained models allow to better consider the respective network architectures, helping isolate parameter changes in the objective, gradient and Hessian from changing data [159]. A steady decrease in the curvature of the loss function has been linked to plasticity loss. To define a measure of curvature, there are different available functions of the Hessian matrix of the objective function [97]. One common measure is the sharpness of local minima, given by the maximum eigenvalue of the Hessian which describes the steepest curvature [69]. However, this sharpness measure is coarse-grained, only giving the magnitude of the vector of maximal curvature, and failing to characterize other directions. Another measure is the drop of effective rank of the Hessian matrix, which counts the effective number of directions in the parameter space that meaningfully influence the curvature of the loss function [98]. Similar to the rank of a matrix representing the number of linearly independent dimensions, the effective rank takes into consideration how each dimension influences the transformation induced by a matrix [120], where a low value implies that the information in most of the dimensions is close to being redundant. Over time, the learning algorithm favors low-rank solutions for the current concept [117], which then serves as the initialization for the next concept, with the effective rank of the representation layer incrementally decreasing. Consequently, this limits the number of solutions that the NN is able to explore at the beginning of a new concept [98]. Furthermore, one can reconcile the geometry of the loss function based on the interpretation of the diagonal entries of the Fisher information matrix of the Hessian rank as an approximation of high local curvature [2].

Probing, similarity and classifier comparison. While these flatness and robustness analyses are useful, it is extremely hard to reliably characterize the optimization landscape, since it can also change as the underlying data distribution changes, even though parameters maintain the same. To alleviate this issue, probing, e.g. freezing the encoder representation and retraining the final linear layer using linear classifiers or k-NN classification, allows for better control [17]. Moreover, one can use representational similarity algorithms to directly compare changes in representations during a concept drift. In this case, the major challenge is overcoming the natural nonalignment between hidden neurons across different NNs [113]. For this purpose, if assuming invariance to orthogonal transformation and isotropic scaling, one can use centered kernel alignment [74]. Regarding the classifier comparison, one can naively perform linear classification trained on the activations of two classifiers both before and after forgetting to measure the performance that can be recovered [115]. Alternatively, one can compare how two classifiers partition the feature space using both high cosine similarity and low L_2 distance as indicators of similarity [17].

Resource efficiency. Beyond latent representations and predictive performance, it is important to measure the computational resource trade-offs, e.g. using the NetScore metric [147] or RAM-Hours [40], the time for both inference and training, and even the energy consumption rate of the algorithm, e.g. accounting for the static leakage power consumed when there is no circuit activity, and the dynamic power dissipated by the circuit, from charging and discharging the capacitor [42]. If one estimates the total execution time as the clock cycle time of the processor multiplied by the clock cycles per instruction and the number of instructions, the resulting energy consumed becomes a function of the energy of each instruction, which can be estimated by analyzing the memory accesses delays, computation requirements (integer and floating point operations), cache accesses, and cache misses [42].

6. Conclusion

In this survey, we explored state-of-the-art methods for on-device training, particularly in the context of classification tasks using neural networks (NNs) and decision trees (DTs). **RQ1** highlighted that the key constraints for on-device training stem from the data architecture (batch vs. stream) and network capacity (cloud vs. edge). These factors influence algorithm design, particularly in terms of data arrival order and execution models. We also identified a clear need for performance evaluation based on multi-criteria metrics, tailored to specific edge applications. **RQ2** examined the building blocks necessary for deploying NNs on edge devices, addressing not only techniques for compressing NNs to fit resource-limited devices, but also challenges such as catastrophic forgetting and data inefficiency. Additionally, we discussed the difficulty NNs face in capturing irregular patterns found in IoT tabular data and operating in open-world settings with minimal supervision. **RQ3** focused on the adaptation of decision trees for on-device training, emphasizing memory-efficient methods, overcoming reliance on heavy supervision, and addressing the issue of fixed sub-spaces as descendant nodes evolve. While NNs can adapt through both dynamic (parameter addition) and fixed (relevant parts activated) architectures, DTs rely more heavily on dynamic adaptations such as structural expansion, pruning, and meta-learning to model the concept history. **RQ4** emphasized that evaluating on-device training strategies requires more than just output-based metrics. A comprehensive evaluation must also account for the model's internal representation quality, assessing how well it supports future learning. Multi-objective metrics, such as probing, robustness analysis, and representational similarity, are crucial for understanding how different algorithms adapt to the varied demands of edge scenarios, considering both hardware limitations and data characteristics.

On-device edge training remains a complex challenge. While NNs show great potential, they require significant optimization to address issues such as resource constraints, catastrophic forgetting, and handling tabular data. In contrast, DTs are more efficient and better suited for resource-limited environments, but their limited expressiveness restricts their ability to capture complex patterns. This creates a trade-off between expressiveness and convergence, where DTs must fully adapt to new concepts, while NNs can integrate new knowledge with less interference. However, as data complexity increases, NNs struggle with slower convergence rates, leading to longer recovery times when adapting to new distributions. The critical challenge now lies in integrating these components into autonomous, resource-limited learning systems, balancing the stability-plasticity and forward-backward transfer trade-offs. Additionally, complementary approaches not covered in this survey, such as federated learning, offer promising directions for scaling model training across multiple edge devices while preserving data privacy and handling heterogeneous distributions.

References

[1] Abati, D., Tomczak, J., Blankevoort, T., Calderara, S., Cucchiara, R., Bejnordi, B.E., 2020. Conditional channel gated networks for task-aware continual learning, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 3931–3940.

[2] Achille, A., Rovere, M., Santoro, S., 2018. Critical learning periods in deep networks. arXiv preprint arXiv:1711.08856 .

[3] Aljundi, R., Belilovsky, E., Tuytelaars, T., Charlin, L., Caccia, M., Lin, M., Page-Caccia, L., 2019. Online continual learning with maximal interfered retrieval, in: Advances in Neural Information Processing Systems, pp. 11849–11860.

[4] Almeida, P.R., Oliveira, L.S., Britto Jr, A.S., Sabourin, R., 2018. Adapting dynamic classifier selection for concept drift. Expert Systems with Applications 104, 67–85.

[5] Anderson, R., Koh, Y.S., Dobbie, G., 2016. Recurring concept meta-learning for evolving data streams, in: Pacific-Asia Conference on Knowledge Discovery and Data Mining, Springer. pp. 3–14.

[6] Ankorion, I., 2005. Change data capture: Efficient etl for real-time bi. Information Management 15.

[7] Arik, S.O., Pfister, T., 2021. Tabnet: Attentive interpretable tabular learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 6679–6687.

[8] Ash, J., Adams, R.P., 2020. On warm-starting neural network training, in: Advances in Neural Information Processing Systems, pp. 3884–3894.

[9] Badirli, S., Liu, X., Xing, Z., Bhowmik, A., Doan, K., Keerthi, S.S., 2020. Gradient boosting neural networks: Grownet. arXiv preprint arXiv:2002.07971 .

[10] Bang, J., Kim, H., Yoo, Y., Ha, J.W., Choi, J., 2021. Rainbow memory: Continual learning with a memory of diverse samples, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8218–8227.

[11] Barddal, J.P., Enembreck, F., 2020. Regularized and incremental decision trees for data streams. Annals of Telecommunications 75, 493–503.

[12] Bifet, A., Gavalda, R., 2009. Adaptive learning from evolving data streams, in: Advances in Intelligent Data Analysis VIII: 8th International Symposium on Intelligent Data Analysis, IDA 2009, Lyon, France, August 31-September 2, 2009. Proceedings 8, Springer. pp. 249–260.

[13] Bonet, D., Montserrat, D.M., Giró-i Nieto, X., Ioannidis, A.G., 2024. Hyperfast: Instant classification for tabular data, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 11114–11123.

[14] Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., Kasneci, G., 2022. Deep neural networks and tabular data: A survey. IEEE transactions on neural networks and learning systems .

[15] Buzzega, P., Boschini, M., Porrello, A., Abati, D., Calderara, S., 2020. Dark experience for general continual learning: a strong, simple baseline. Advances in neural information processing systems 33, 15920–15930.

[16] Caccia, M., Belilovsky, E., Caccia, M., Pineau, J., 2022. New insights on reducing abrupt representation change in online continual learning. arXiv preprint arXiv:2203.06389 .

[17] Cha, S., Shim, D., Kim, H., Lee, M., Lee, H., Moon, T., 2022. Towards more objective evaluation of class incremental learning: Representation learning perspective. arXiv preprint arXiv:2206.08101 .

[18] Chaudhry, A., Ranzato, M., Rohrbach, M., Elhoseiny, M., 2018. Efficient lifelong learning with a-gem, in: International Conference on Learning Representations.

[19] Chaudhry, A., Rohrbach, M., Elhoseiny, M., Ajanthan, T., Dokania, P.K., Torr, P.H., Ranzato, M., 2019. On tiny episodic memories in continual learning. arXiv preprint arXiv:1902.10486 .

[20] Cheung, B., Terekhov, A., Chen, Y., Agrawal, P., Olshausen, B., 2019. Superposition of many models into one. Advances in Neural Information Processing Systems 32.

[21] da Costa, V.G.T., de Leon Ferreira, A.C.P., Junior, S.B., et al., 2018. Strict very fast decision tree: a memory conservative algorithm for data stream mining. Pattern Recognition Letters 116, 22–28.

[22] Courbariaux, M., Bengio, Y., David, J.P., 2015. Binaryconnect: Training deep neural networks with binary weights during propagations, in: Advances in neural information processing systems, pp. 3123–3131.

[23] De Lange, M., Tuytelaars, T., 2021. Continual prototype evolution: Learning online from non-stationary data streams. Proceedings of the IEEE/CVF International Conference on Computer Vision , 8250–8259.

[24] De Rosa, R., Cesa-Bianchi, N., 2017. Confidence decision trees via online and active learning for streaming data. Journal of Artificial Intelligence Research 60, 1031–1055.

[25] Debnath, B.K., Lilja, D.J., Mokbel, M.F., 2008. SARD: A statistical approach for ranking database tuning parameters, in: 2008 IEEE 24th International Conference on Data Engineering Workshop, IEEE. IEEE. pp. 11–18.

[26] Denton, E.L., Zaremba, W., Bruna, J., LeCun, Y., Fergus, R., 2014. Exploiting linear structure within convolutional networks for efficient evaluation, in: Advances in neural information processing systems.

[27] Dohare, S., Hernandez-Garcia, A., Lacoste, A., Weiss, M., 2021. Continual backprop: Stochastic gradient descent with persistent randomness, in: International Conference on Machine Learning, pp. 2660–2670.

[28] Domingos, P., Hulten, G., 2000. Mining high-speed data streams, in: Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 71–80.

[29] Domingos, P.M., Hulten, G., 2001. Catching up with the data: Research issues in mining data streams, in: 2001 ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, Santa Barbara, CA, USA.

[30] Dong, X., Chen, S., Pan, S., 2017. Learning to prune deep neural networks via layer-wise optimal brain surgeon, in: Advances in Neural Information Processing Systems, pp. 4857–4867.

[31] Ducange, P., Marcelloni, F., Pecori, R., et al., 2021. Fuzzy hoeffding decision tree for data stream classification. Int. J. Comput. Intell. Syst. 14, 946–964.

[32] Duda, P., Jaworski, M., Pietruczuk, L., Rutkowski, L., 2014. A novel application of hoeffding's inequality to decision trees construction for data streams, in: 2014 International Joint Conference on Neural Networks (IJCNN), IEEE. pp. 3324–3330.

[33] Faria, E.R.d., Gama, J., Carvalho, A.C.P.d.L.F., 2016. Minas: multiclass learning algorithm for novelty detection in data streams. Data Mining and Knowledge Discovery 30, 640–680.

[34] Frias-Blanco, I., del Campo-Ávila, J., Ramos-Jimenez, G., Morales-Bueno, R., Ortiz-Diaz, A., Caballero-Mota, Y., 2014. Online and non-parametric drift detection methods based on hoeffding's bounds. IEEE Transactions on Knowledge and Data Engineering 27, 810–823.

[35] Frosst, N., Hinton, G., 2017. Distilling a neural network into a soft decision tree. arXiv preprint arXiv:1711.09784 .

[36] Gama, J., Medas, P., Rocha, R., 2004. Forest trees for on-line data, in: Proceedings of the 2004 ACM symposium on Applied computing, pp. 632–636.

[37] Gama, J., Medas, P., Rodrigues, P., 2005. Learning decision trees from dynamic data streams, in: Proceedings of the 2005 ACM Symposium on Applied computing, pp. 573–577.

[38] Gama, J., Pinto, C., 2006. Discretization from data streams: applications to histograms and data mining, in: Proceedings of the 2006 ACM symposium on Applied computing, pp. 662–667.

[39] Gama, J., Rocha, R., Medas, P., 2003. Accurate decision trees for mining high-speed data streams, in: Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 523–528.

[40] Gama, J., Sebastiao, R., Rodrigues, P.P., 2013. On evaluating stream learning algorithms. Machine learning 90, 317–346.

[41] Garcia-Martin, E., Bifet, A., Lavesson, N., König, R., Linusson, H., 2022. Green accelerated hoeffding tree. arXiv preprint arXiv:2205.03184 .

[42] García-Martín, E., Lavesson, N., Grahn, H., Casalicchio, E., Boeva, V., 2018. Hoeffding trees with nmin adaptation, in: 2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA), IEEE. pp. 70–79.

[43] Geng, C., Huang, S.j., Chen, S., 2020. Recent advances in open set recognition: A survey. IEEE transactions on pattern analysis and machine intelligence 43, 3614–3631.

[44] Gomes, H.M., Bifet, A., Read, J., Barddal, J.P., Enembreck, F., Pfharinger, B., Holmes, G., Abdessalem, T., 2017. Adaptive random forests for evolving data stream classification. Machine Learning 106, 1469–1495.

[45] Gorishniy, Y., Rubachev, I., Babenko, A., 2022. On embeddings for numerical features in tabular deep learning. Advances in Neural Information Processing Systems 35, 24991–25004.

[46] Gözüaçık, Ö., Can, F., 2021. Concept learning using one-class classifiers for implicit drift detection in evolving data streams. Artificial Intelligence Review 54, 3725–3747.

[47] Grinsztajn, L., Oyallon, E., Varoquaux, G., 2022. Why do tree-based models still outperform deep learning on tabular data? arXiv preprint arXiv:2207.08815 .

[48] Gunasekara, N., Pfahringer, B., Gomes, H., Bifet, A., 2024. Gradient boosted trees for evolving data streams. Machine Learning 113, 3325–3352.

[49] Guo, Y., Liu, B., Zhao, D., 2023. Dealing with cross-task class discrimination in online continual learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 20446–20455.

[50] Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y., 2022. Adbench: Anomaly detection benchmark. Advances in Neural Information Processing Systems 35, 32142–32159.

[51] Han, S., Pool, J., Tran, J., Dally, W., 2015. Learning both weights and connections for efficient neural network, in: Advances in neural information processing systems, pp. 1135–1143.

[52] Haque, A., Khan, L., Baron, M., 2016. Sand: Semi-supervised adaptive novel class detection and classification over data stream, in: Proceedings of the AAAI Conference on Artificial Intelligence.

[53] Harrison, J., Sharma, A., Finn, C., Pavone, M., 2020. Continuous meta-learning without tasks. Advances in neural information processing systems 33, 17571–17581.

[54] Hayes, T.L., Kafle, K., Shrestha, R., Acharya, M., Kanan, C., 2020. Remind your neural network to prevent catastrophic forgetting. European Conference on Computer Vision , 466–483.

[55] Hess, T., Verwimp, E., van de Ven, G.M., Tuytelaars, T., 2023. Knowledge accumulation in continually learned representations and the issue of feature forgetting. arXiv preprint arXiv:2304.00933 .

[56] Heyden, M., Gomes, H.M., Fouché, E., Pfahringer, B., Böhm, K., 2024. Leveraging plasticity in incremental decision trees, in: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer. pp. 38–54.

[57] Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S.B., Schirrmeister, R.T., Hutter, F., 2025. Accurate predictions on small data with a tabular foundation model. Nature 637, 319–326.

[58] Holmes, G., Richard, K., Pfahringer, B., 2005. Tie-breaking in hoeffding trees, ECML/PKDD.

[59] Howard, A.G., 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 .

[60] Huang, X., Khetan, A., Cvitkovic, M., Karnin, Z., 2020. Tabtransformer: Tabular data modeling using contextual embeddings. arXiv preprint arXiv:2012.06678 .

[61] Hulten, G., Spencer, L., Domingos, P., 2001. Mining time-changing data streams, in: Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 97–106.

[62] Iandola, F.N., 2016. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and< 0.5 mb model size. arXiv preprint arXiv:1602.07360 .

[63] Ikonomovska, E., Gama, J., Džeroski, S., 2011. Learning model trees from evolving data streams. Data mining and knowledge discovery 23, 128–168.

[64] Jaderberg, M., Vedaldi, A., Zisserman, A., 2014. Speeding up convolutional neural networks with low rank expansions. arXiv preprint arXiv:1405.3866 .

[65] Jeffares, A., Liu, T., Crabbé, J., Imrie, F., van der Schaar, M., 2023. Tangos: Regularizing tabular neural networks through gradient orthogonalization and specialization. arXiv preprint arXiv:2303.05506 .

[66] Katakis, I., Tsoumakas, G., Vlahavas, I., 2010. Tracking recurring contexts using ensemble classifiers: an application to email filtering. Knowledge and Information Systems 22, 371–391.

[67] Katzir, O., Elidan, G., El-Yaniv, R., 2021. Net-dnf: Effective deep modeling of tabular data, in: International Conference on Learning Representations.

[68] Kemker, R., McClure, M., Abitino, A., Hayes, T.L., Kanan, C., 2018. Measuring catastrophic forgetting in neural networks. Proceedings of the AAAI Conference on Artificial Intelligence 32.

[69] Keskar, N.S., Mudigere, D., Nocedal, J., Smelyanskiy, M., Tang, P.T.P., 2017. On large-batch training for deep learning: Generalization gap and sharp minima, in: International Conference on Learning Representations.

[70] Kim, G., Xiao, C., Konishi, T., Ke, Z., Liu, B., 2022. A theoretical study on solving continual learning. Advances in neural information processing systems 35, 5065–5079.

[71] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al., 2017. Overcoming catastrophic forgetting in neural networks. Proceedings of the national academy of sciences 114, 3521–3526.

[72] Kirsch, L., Kunze, J., Barber, D., 2018. Modular networks: Learning to decompose neural computation, in: Advances in Neural Information Processing Systems, pp. 2408–2418.

[73] Knoblauch, J., Husain, H., Diethe, T., 2020. Optimal continual learning has perfect memory and is np-hard. arXiv preprint arXiv:2006.05188 .

[74] Kornblith, S., Norouzi, M., Lee, H., Hinton, G., 2019. Similarity of neural network representations revisited, in: International Conference on Machine Learning, PMLR. pp. 3519–3529.

[75] Kreps, J., 2014. Questioning the lambda architecture. the lambda architecture has its merits, but alternatives are worth exploring. O'Reilly Media URL: https://www.oreilly.com/ideas/questioning-the-lambda-architecture.

[76] Kumar, S., Marklund, H., Van Roy, B., 2023. Maintaining plasticity via regenerative regularization. arXiv preprint arXiv:2308.11958 .

[77] Lefa, M., Hatem, A., Salem, R., 2022. Enhancement of very fast decision tree for data stream mining. Studies in Informatics and Control 31, 49–60.

[78] Lewandowski, A., Tanaka, H., Schuurmans, D., Machado, M.C., 2023. Curvature explains loss of plasticity .

[79] Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H.P., 2016. Pruning filters for efficient convnets. arXiv preprint arXiv:1608.08710 .

[80] Lin, J., 2017. The lambda and the kappa. IEEE Internet Computing 21, 60–66.

[81] Lindstrom, P., Mac Namee, B., Delany, S.J., 2013. Drift detection using uncertainty distribution divergence. Evolving Systems 4, 13–25.

[82] Liu, H., Simonyan, K., Yang, Y., 2018. Darts: Differentiable architecture search, in: International Conference on Learning Representations.

[83] Liu, S., Biderman, S., Schoelkopf, H., Downey, C., Prashanth, R., Ramasesh, V.V., Schmidt, L., 2023. Small-scale proxies for large-scale transformer training instabilities. arXiv preprint arXiv:2305.14718 .

[84] Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., Zhang, C., 2017. Learning efficient convolutional networks through network slimming, in: Proceedings of the IEEE international conference on computer vision, pp. 2736–2744.

[85] Liu, Z., Sun, M., Zhou, T., Huang, G., Darrell, T., 2019. Rethinking the value of network pruning, in: International Conference on Learning Representations.

[86] Losing, V., Wersing, H., Hammer, B., 2018. Enhancing very fast decision trees with local split-time predictions, in: 2018 IEEE international conference on data mining (ICDM), IEEE. pp. 287–296.

[87] Louizos, C., Welling, M., Kingma, D.P., 2017. Learning sparse neural networks through l_0 regularization. arXiv preprint arXiv:1712.01312 .

[88] Lourenço, A., Gama, J.a., Xing, E.P., Marreiros, G., 2025a. In-context learning of evolving data streams with tabular foundational models. arXiv preprint arXiv:2502.16840 .

[89] Lourenço, A., Rodrigo, J.a., Gama, J.a., Marreiros, G., 2025b. Dfdt: Dynamic fast decision tree for iot data stream mining on edge devices. arXiv preprint arXiv:2502.14011 .

[90] Lyle, C., Schuurmans, D., Gal, Y., Machado, M.C., 2024. Disentangling the causes of plasticity loss in neural networks. arXiv preprint arXiv:2402.18762 .

[91] Lyle, C., Zheng, Z., Nikishin, E., Pires, B.A., Pascanu, R., Dabney, W., 2023. Understanding plasticity in neural networks, in: International Conference on Machine Learning, PMLR. pp. 23190–23211.

[92] Manapragada, C., Webb, G.I., Salehi, M., 2018. Extremely fast decision tree, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1953–1962.

[93] Masud, M.M., Chen, Q., Khan, L., Aggarwal, C.C., Gao, J., Han, J., Srivastava, A., Oza, N.C., 2013. Classification and novel class detection of data streams in a dynamic feature space. IEEE Transactions on Knowledge and Data Engineering 26, 965–979.

[94] Masud, M.M., Gao, J., Khan, L., Han, J., Thuraisingham, B., 2011. Classification and novel class detection in concept-drifting data streams under time constraints, in: IEEE Transactions on Knowledge and Data Engineering, IEEE. pp. 859–874.

[95] Merrill, W., Goldberg, Y., Schwartz, R., Smith, N.A., 2023. Effects of parameter norm growth during transformer training: Inductive bias from gradient descent. Transactions of the Association for Computational Linguistics 11, 121–136.

[96] Mirkhan, M., Amir Haeri, M., Meybodi, M.R., 2021. Analytical split value calculation for numerical attributes in hoeffding trees with misclassification-based impurity. Annals of Data Science 8, 645–665.

[97] Mirzadeh, S.I., Farajtabar, M., Pascanu, R., Ghasemzadeh, H., 2020. Understanding the role of training regimes in continual learning, in: Advances in Neural Information Processing Systems.

[98] Mirzadeh, S.I., Farajtabar, M., Pascanu, R., Ghasemzadeh, H., 2023. Directions of curvature as an explanation for loss of plasticity. arXiv preprint arXiv:2302.03044 .

[99] Molchanov, P., Mallya, A., Tyree, S., Frosio, I., Kautz, J., 2019. Importance estimation for neural network pruning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11264–11272.

[100] Molchanov, P., Tyree, S., Karras, T., Aila, T., Kautz, J., 2016. Pruning convolutional neural networks for resource efficient inference. arXiv preprint arXiv:1611.06440 .

[101] Mouss, H., Mouss, D., Mouss, N., Sefouhi, L., 2004. Test of page-hinckley, an approach for fault detection in an agro-alimentary production system. 2004 5th Asian Control Conference (IEEE Cat. No. 04EX904) 2, 815–818.

[102] Nagel, M., Baalen, M.v., Blankevoort, T., Welling, M., 2019. Data-free quantization through weight equalization and bias correction, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 1325–1334.

[103] Nguyen, C.V., Li, Y., Bui, T.D., Turner, R.E., 2018. Variational continual learning, in: International Conference on Learning Representations.

[104] Nikishin, E., Oh, J., Ostrovski, G., Lyle, C., Pascanu, R., Dabney, W., Barreto, A., 2024. Deep reinforcement learning with plasticity injection, in: Advances in Neural Information Processing Systems.

[105] Núñez, M., Fidalgo, R., Morales, R., 2007. Learning in environments with unknown dynamics: Towards more robust concept learners. Journal of Machine Learning Research 8.

[106] Ostapenko, O., Suris, D., Szabó, A., Mikolov, T., 2022. Attention for compositional modularity, in: NeurIPS'22 Workshop on All Things Attention: Bridging Different Perspectives on Attention.

[107] von Oswald, J., Henning, C., Sacramento, J., Grewe, B.F., 2019. Continual learning with hypernetworks. arXiv preprint arXiv:1906.00695 .

[108] Oza, N.C., Russell, S., 2001. Experimental comparisons of online and batch versions of bagging and boosting, in: Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 359–364.

[109] Park, S., Yoo, J., Cho, D., Kim, J., Yoon, S., 2020. Lookahead: A far-sighted alternative of magnitude-based pruning, in: International Conference on Learning Representations.

[110] Pietruczuk, L., Rutkowski, L., Jaworski, M., Duda, P., 2017. A method for automatic adjustment of ensemble size in stream data mining. Information Sciences 381, 46–54.

[111] Popov, S., Morozov, S., Babenko, A., 2019. Neural oblivious decision ensembles for deep learning on tabular data. arXiv preprint arXiv:1909.06312 .

[112] Prabhu, A., Torr, P.H., Dokania, P.K., 2020. Gdumb: A simple approach that questions our progress in continual learning, in: European Conference on Computer Vision, Springer. pp. 524–540.

[113] Raghu, M., Gilmer, J., Yosinski, J., Sohl-Dickstein, J., 2017. Svcca: Singular vector canonical correlation analysis for deep learning dynamics and interpretability, in: Advances in Neural Information Processing Systems, pp. 6076–6085.

[114] Ramasesh, V.V., Dyer, E., Raghu, M., 2020a. Anatomy of catastrophic forgetting: Hidden representations and task semantics. arXiv preprint arXiv:2007.07400 .

[115] Ramasesh, V.V., Dyer, E., Raghu, M., 2020b. Anatomy of catastrophic forgetting: Hidden representations and task semantics. arXiv preprint arXiv:2007.07400 .

[116] Razavi-Far, R., Hallaji, E., Saif, M., Ditzler, G., 2018. A novelty detector and extreme verification latency model for nonstationary environments. IEEE Transactions on Industrial Electronics 66, 561–570.

[117] Razin, N., Cohen, N., 2020. Implicit regularization in deep learning may not be explainable by norms. Advances in Neural Information Processing Systems 33, 2803–2814.

[118] dos Reis, D.M., Flach, P., Matwin, S., Batista, G., 2016. Fast unsupervised online drift detection using incremental kolmogorov-smirnov test, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1545–1554.

[119] Rosenbaum, C., Klinger, T., Riemer, M., 2019. Routing networks and the challenges of modular and compositional computation. arXiv preprint arXiv:1904.12774 .

[120] Roy, O., Vetterli, M., 2007. The effective rank: A measure of effective dimensionality, in: 2007 15th European Signal Processing Conference, IEEE. pp. 606–610.

[121] Rusu, A.A., Rabinowitz, N.C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., Hadsell, R., 2016. Progressive neural networks. arXiv preprint arXiv:1606.04671 .

[122] Rutkowski, L., Jaworski, M., Pietruczuk, L., Duda, P., 2014. A new method for data stream mining based on the misclassification error. IEEE transactions on neural networks and learning systems 26, 1048–1059.

[123] Rutkowski, L., Pietruczuk, L., Duda, P., Jaworski, M., 2012. Decision trees for mining data streams based on the mcdiarmid's bound. IEEE Transactions on Knowledge and Data Engineering 25, 1272–1279.

[124] Salehi, M., Sadjadi, N., Baghshah, S., Rabiee, H.R., Rohban, M.H., 2021. A unified survey on anomaly, novelty, open-set, and out-of-distribution detection: Solutions and future challenges. arXiv preprint arXiv:2110.14051 .

[125] Schizas, N., Karras, A., Karras, C., Sioutas, S., 2022. Tinyml for ultra-low power ai and large scale iot deployments: A systematic review. Future Internet 14, 363.

[126] Schwarz, J., Luketina, J., Czarnecki, W.M., Grabska-Barwinska, A., Teh, Y.W., Pascanu, R., Hadsell, R., 2018. Progress & compress: A scalable framework for continual learning. International Conference on Machine Learning , 4528–4537.

[127] Serra, J., Suris, D., Miron, M., Karatzoglou, A., 2018. Overcoming catastrophic forgetting with hard attention to the task. International Conference on Machine Learning , 4548–4557.

[128] Sethi, T.S., Kantardzic, M., 2015. Don't pay for validation: Detecting drifts from unlabeled data using margin density. Procedia Computer

Science 53, 103–112.
[129] Shaker, A., Hüllermeier, E., 2015. Recovery analysis for adaptive learning from non-stationary data streams: Experimental design and case study. Neurocomputing 150, 250–264.
[130] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., Dean, J., 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, in: International Conference on Learning Representations.
[131] Shin, H., Lee, J.K., Kim, J., Kim, J., 2017. Continual learning with deep generative replay, in: Advances in neural information processing systems, pp. 2990–2999.
[132] Shwartz-Ziv, R., Armon, A., 2022. Tabular data: Deep learning is not all you need. Information Fusion 81, 84–90.
[133] Sokar, G., Agarwal, R., Castro, P.S., Evci, U., 2023. The dormant neuron phenomenon in deep reinforcement learning, in: International Conference on Machine Learning, PMLR. pp. 32145–32168.
[134] Somepalli, G., Goldblum, M., Schwarzschild, A., Bruss, C.B., Goldstein, T., 2021. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. arXiv preprint arXiv:2106.01342 .
[135] Spinosa, E.J., de Carvalho, A.C., Gama, J., 2009. Olindda: A cluster-based approach for detecting novelty and concept drift in data streams. Expert Systems with Applications 36, 4370–4382.
[136] Sun, H., Ren, X., Liang, S., Zhang, Y., Jiang, C., Yin, W., 2020. A survey on dynamic neural networks for natural language processing. arXiv preprint arXiv:2006.00693 .
[137] Sun, J., Jia, H., Hu, B., Huang, X., Zhang, H., Wan, H., Zhao, X., 2021. Speeding up very fast decision tree with low computational cost, in: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence, pp. 1272–7278.
[138] Tan, C.W., Bergmeir, C., Petitjean, F., Webb, G.I., 2020. Converting tabular data into images for deep learning with convolutional neural networks, in: 2020 International Joint Conference on Neural Networks (IJCNN), IEEE. pp. 1–8.
[139] Van Dongen, G., Van den Poel, D., 2020. Evaluation of stream processing frameworks. IEEE Transactions on Parallel and Distributed Systems 31, 1845–1858.
[140] Verwimp, E., Ben-David, S., Bethge, M., Cossu, A., Gepperth, A., Hayes, T.L., Hüllermeier, E., Kanan, C., Kudithipudi, D., Lampert, C.H., et al., 2023. Continual learning: Applications and the road forward. arXiv preprint arXiv:2311.11908 .
[141] Wang, T., Zhu, J.Y., Torralba, A., Efros, A.A., 2018. Dataset distillation. arXiv preprint arXiv:1811.10959 .
[142] Wang, Y., Yao, Q., Kwok, J.T., Ni, L.M., 2020. Generalizing from a few examples: A survey on few-shot learning. ACM Computing Surveys (CSUR) 53, 1–34.
[143] Wang, Z., Sun, J., 2022. Transtab: Learning transferable tabular transformers across tables. Advances in Neural Information Processing Systems 35, 2902–2915.
[144] Warden, P., Situnayake, D., 2019. Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers. O'Reilly Media.
[145] Warren, J., Marz, N., 2015. Big Data: Principles and best practices of scalable realtime data systems. Simon and Schuster.
[146] Wen, W., Wu, C., Wang, Y., Chen, Y., Li, H., 2016. Learning structured sparsity in deep neural networks, in: Advances in neural information processing systems.
[147] Wong, A., 2019. Netscore: towards universal metrics for large-scale performance analysis of deep neural networks for practical on-device edge usage, in: International Conference on Image Analysis and Recognition, Springer. pp. 15–26.
[148] Wortsman, M., Ramanujan, V., Liu, R., Kembhavi, A., Rastegari, M., Yosinski, J., Farhadi, A., 2020. Supermasks in superposition, in: Advances in Neural Information Processing Systems, pp. 15173–15184.
[149] Wu, C., Wu, F., Qi, T., Huang, Y., Xie, X., 2021. Fastformer: Additive attention can be all you need. arXiv preprint arXiv:2108.09084 .
[150] Wu, J., Chen, S., Zhao, Q., Sergazinov, R., Li, C., Liu, S., Zhao, C., Xie, T., Guo, H., Ji, C., et al., 2024. Switchtab: Switched autoencoders are effective tabular learners, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 15924–15933.
[151] Xian, Y., Lampert, C.H., Schiele, B., Akata, Z., 2018. Zero-shot learning—a comprehensive evaluation of the good, the bad and the ugly. IEEE transactions on pattern analysis and machine intelligence 41, 2251–2265.
[152] Xu, S., Wang, J., 2016. Mining recurring concept drifts with limited labeled streaming data. ACM Transactions on Intelligent Systems and Technology (TIST) 7, 1–32.
[153] Yamada, Y., Lindenbaum, O., Negahban, S., Kluger, Y., 2020. Feature selection using stochastic gates, in: International conference on machine learning, PMLR. pp. 10648–10659.
[154] Yang, H., Fong, S., 2011a. Moderated vfdt in stream mining using adaptive tie threshold and incremental pruning, in: Data Warehousing and Knowledge Discovery: 13th International Conference, DaWaK 2011, Toulouse, France, August 29-September 2, 2011. Proceedings 13, Springer. pp. 471–483.
[155] Yang, H., Fong, S., 2011b. Optimized very fast decision tree with balanced classification accuracy and compact tree size, in: The 3rd international conference on data mining and intelligent information technology applications, IEEE. pp. 57–64.
[156] Ye, H., Fan, W., Song, X., Zheng, S., Zhao, H., Guo, D., Chang, Y., 2024. Ptarl: Prototype-based tabular representation learning via space calibration. arXiv preprint arXiv:2407.05364 .
[157] Yoon, J., Yang, E., Lee, J., Hwang, S.J., 2018. Dynamically expandable networks. arXiv preprint arXiv:1708.01547 .
[158] Yoon, J., Zhang, Y., Jordon, J., Van der Schaar, M., 2020. Vime: Extending the success of self-and semi-supervised learning to tabular domain. Advances in Neural Information Processing Systems 33, 11033–11043.
[159] Zhang, C., Bengio, S., Singer, Y., 2023. Are all layers created equal? Journal of Machine Learning Research 24, 1–52.
[160] Zhang, L., Bao, C., Ma, K., 2021. Self-distillation: Towards efficient and compact neural networks. IEEE Transactions on Pattern Analysis and Machine Intelligence 44, 4388–4403.
[161] Zhang, L., Song, J., Gao, A., Chen, J., Bao, C., Ma, K., 2019. Be your own teacher: Improve the performance of convolutional neural networks via self distillation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 3713–3722.
[162] Zhang, W., Bifet, A., 2020. Feat: A fairness-enhancing and concept-adapting decision tree classifier, in: Discovery Science: 23rd International Conference, DS 2020, Thessaloniki, Greece, October 19–21, 2020, Proceedings 23, Springer. pp. 175–189.
[163] Zhang, X., Zhou, X., Lin, M., Sun, J., 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 6848–6856.
[164] Zhu, B., Shi, X., Erickson, N., Li, M., Karypis, G., Shoaran, M., 2023. Xtab: Cross-table pretraining for tabular transformers. arXiv preprint arXiv:2305.06090 .
[165] Žliobaite, I., 2010. Change with delayed labeling: When is it detectable?, in: 2010 IEEE international conference on data mining workshops, IEEE. pp. 843–850.